LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara 15 Konsep API dan JSON

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan aplikasi web membutuhkan cara untuk bertukar data antara satu aplikasi dengan aplikasi lainnya. Salah satu teknologi yang digunakan adalah Application Programming Interface (API), yang memungkinkan aplikasi mengakses data melalui aturan komunikasi tertentu.

JSON (JavaScript Object Notation) merupakan format pertukaran data yang ringan dan mudah dibaca. Dalam pengembangan web menggunakan PHP, JSON dapat digunakan untuk mengirimkan data dari database kepada pengguna atau aplikasi lain.

Oleh karena itu, praktikum ini dilakukan untuk memahami konsep API dan JSON serta menerapkannya pada sistem informasi akademik menggunakan PHP dan MySQL.

1.2 Rumusan Masalah

1. Apa yang dimaksud dengan API dan JSON?
2. Bagaimana membuat API menggunakan PHP dan database MySQL?
3. Bagaimana mengambil dan menambahkan data mahasiswa melalui API?

1.3 Tujuan Praktikum

1. Memahami konsep API dan JSON.
2. Membuat API untuk mengakses data mahasiswa.
3. Menerapkan metode GET dan POST.
4. Menguji respons API menggunakan browser dan Postman.

1.4 Manfaat Praktikum

Praktikum ini memberikan pemahaman mengenai pertukaran data menggunakan API dan JSON serta meningkatkan kemampuan dalam mengembangkan layanan web yang terhubung dengan database.

BAB 2. LANDASAN TEORI

2.1 Application Programming Interface (API)

API merupakan antarmuka yang memungkinkan dua aplikasi atau lebih berkomunikasi dan bertukar data. Pada aplikasi web, API dapat digunakan untuk mengambil, menambahkan, memperbarui, atau menghapus data melalui permintaan HTTP.

2.2 JavaScript Object Notation (JSON)

JSON adalah format pertukaran data yang menggunakan struktur pasangan kunci dan nilai. Format ini mudah dibaca serta dapat digunakan oleh berbagai bahasa pemrograman.

Contoh format JSON:

```
{
    "success": true,
    "message": "Data berhasil diambil",
    "data": [
        {
            "id": 1,
            "nim": "23001",
            "nama": "Azira Mutia Gani",
            "email": "azira@gmail.com",
            "angkatan": "2025",
            "prodi": "Teknik Informatika",
            "status": "lulus",
            "dosen_id": 1,
            "prodi_id": 1
        }
    ]
}
```

2.3 HTTP Method

HTTP Method digunakan untuk menentukan tindakan yang dilakukan ter-hadap sumber daya pada API.

a. GET: mengambil data.
b. POST: menambahkan data.
c. PUT: memperbarui seluruh data.
d. PATCH: memperbarui sebagian data.
e. DELETE: menghapus data.

Pada praktikum ini, metode yang digunakan adalah GET dan POST.

2.4 PHP

PHP merupakan bahasa pemrograman sisi server yang digunakan untuk mengolah permintaan pengguna, berkomunikasi dengan database, dan menghasilkan respons API.

PHP menyediakan fungsi json_encode() untuk mengubah data menjadi JSON dan json_decode() untuk membaca data JSON.

2.5 MySQL

MySQL merupakan sistem manajemen basis data relasional yang digunakan untuk menyimpan dan mengelola data. Pada praktikum ini, MySQL digunakan untuk menyimpan data mahasiswa dalam sistem informasi akademik.

2.6 PDO (PHP Data Objects)

PDO merupakan fitur PHP untuk mengakses database. PDO mendukung penggunaan prepared statement sehingga nilai yang dimasukkan ke dalam query dapat diproses dengan lebih aman.

2.7 Endpoint API

Endpoint merupakan alamat yang digunakan untuk mengakses layanan API. Pada praktikum ini, endpoint digunakan untuk mengambil seluruh data mahasiswa, mengambil data berdasarkan ID, dan menambahkan data mahasiswa.

2.8 HTTP Response dan Status Code

HTTP response merupakan jawaban yang dikirimkan server setelah menerima permintaan. Status code menunjukkan hasil pemrosesan permintaan, misalnya:

a) 200 OK: permintaan berhasil.
b) 201 Created: data berhasil dibuat.
c) 400 Bad Request: permintaan tidak valid.
d) 404 Not Found: data tidak ditemukan.
e) 405 Method Not Allowed: metode tidak diizinkan.
f) 409 Conflict: terjadi konflik data.
g) 422 Unprocessable Content: data tidak memenuhi validasi.
h) 500 Internal Server Error: terjadi kesalahan pada server.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktikum

3.1.1 Implementasi API Mahasiswa

Implementasi dilakukan pada project si-akademik15 menggunakan PHP dan database MySQL si_akademik7. File API dibuat pada api/mahasiswa.php untuk memproses permintaan data mahasiswa dan memberikan respons dalam format JSON.

```php
<?php

header('Content-Type: application/json; charset=utf-8');

try {
    $pdo = new PDO(
        'mysql:host=localhost;dbname=si_akademik7;char-
set=utf8mb4',
        'root',
        ''
    );

    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEP-
TION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
PDO::FETCH_ASSOC);

    $method = $_SERVER['REQUEST_METHOD'];

    // GET: mengambil data mahasiswa
    if ($method === 'GET') {
        if (isset($_GET['id'])) {
            $id = filter_var($_GET['id'],
FILTER_VALIDATE_INT);

            if ($id === false || $id === null || $id < 1) {
                http_response_code(400);
                echo json_encode([
                    'success' => false,
                    'message' => 'ID mahasiswa tidak valid'
                ]);
                exit;
            }
```

```
            $stmt = $pdo->prepare(
                'SELECT * FROM mahasiswa WHERE id = :id'
            );
            $stmt->execute(['id' => $id]);
            $data = $stmt->fetch();

            if (!$data) {
                http_response_code(404);
                echo json_encode([
                    'success' => false,
                    'message' => 'Mahasiswa tidak ditemukan'
                ]);
                exit;
            }

            echo json_encode([
                'success' => true,
                'message' => 'Data mahasiswa berhasil diam-
bil',
                'data' => $data
            ], JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);

        } else {
            $stmt = $pdo->query(
                'SELECT * FROM mahasiswa ORDER BY id ASC'
            );

            echo json_encode([
                'success' => true,
                'message' => 'Data berhasil diambil',
                'data' => $stmt->fetchAll()
            ], JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
        }

    // POST: menambahkan data mahasiswa
    } elseif ($method === 'POST') {
        $json = file_get_contents('php://input');
        $input = json_decode($json, true);

        if (json_last_error() !== JSON_ERROR_NONE || !is_ar-
ray($input)) {
            http_response_code(400);
            echo json_encode([
                'success' => false,
                'message' => 'Format JSON tidak valid'
            ]);
            exit;
```

```
        }

        $nim = trim($input['nim'] ?? '');
        $nama = trim($input['nama'] ?? '');
        $email = trim($input['email'] ?? '');
        $prodiId = filter_var(
            $input['prodi_id'] ?? null,
            FILTER_VALIDATE_INT
        );
        $angkatan = filter_var(
            $input['angkatan'] ?? null,
            FILTER_VALIDATE_INT
        );
        $status = trim($input['status'] ?? 'aktif');
        $dosenId = $input['dosen_id'] ?? null;

        // Validasi data wajib
        if (
            $nim === '' ||
            $nama === '' ||
            $prodiId === false || $prodiId === null ||
$prodiId < 1 ||
            $angkatan === false || $angkatan === null ||
            $angkatan < 1901 || $angkatan > 2155
        ) {
            http_response_code(422);
            echo json_encode([
                'success' => false,
                'message' => 'NIM, nama, prodi_id, dan
angkatan wajib diisi dengan benar'
            ]);
            exit;
        }

        if (strlen($nim) > 20 || strlen($nama) > 100) {
            http_response_code(422);
            echo json_encode([
                'success' => false,
                'message' => 'NIM maksimal 20 karakter dan
nama maksimal 100 karakter'
            ]);
            exit;
        }

        // Email boleh kosong atau null
        if ($email !== '' && !filter_var($email,
FILTER_VALIDATE_EMAIL)) {
```

```
            http_response_code(422);
            echo json_encode([
                'success' => false,
                'message' => 'Format email tidak valid'
            ]);
            exit;
        }

        if ($email === '') {
            $email = null;
        }

        if (!in_array($status, ['aktif', 'cuti', 'lulus'],
true)) {
            http_response_code(422);
            echo json_encode([
                'success' => false,
                'message' => 'Status harus aktif, cuti, atau
lulus'
            ]);
            exit;
        }

        // Validasi dosen_id jika diisi
        if ($dosenId === '' || $dosenId === null) {
            $dosenId = null;
        } else {
            $dosenId = filter_var($dosenId,
FILTER_VALIDATE_INT);

            if ($dosenId === false || $dosenId < 1) {
                http_response_code(422);
                echo json_encode([
                    'success' => false,
                    'message' => 'dosen_id tidak valid'
                ]);
                exit;
            }
        }

        // Periksa apakah NIM sudah digunakan
        $check = $pdo->prepare(
            'SELECT id FROM mahasiswa WHERE nim = :nim'
        );
        $check->execute(['nim' => $nim]);

        if ($check->fetch()) {
```

```
            http_response_code(409);
            echo json_encode([
                'success' => false,
                'message' => 'NIM sudah terdaftar'
            ]);
            exit;
        }

        // Pastikan program studi tersedia
        $checkProdi = $pdo->prepare(
            'SELECT id FROM prodi WHERE id = :id'
        );
        $checkProdi->execute(['id' => $prodiId]);

        $prodiData = $checkProdi->fetch();

        if (!$prodiData) {
            http_response_code(422);
            echo json_encode([
                'success' => false,
                'message' => 'Program studi tidak ditemukan'
            ]);
            exit;
        }

        // Ambil nama program studi karena kolom
mahasiswa.prodi masih wajib diisi.
        $stmtNamaProdi = $pdo->prepare(
            'SELECT nama FROM prodi WHERE id = :id'
        );
        $stmtNamaProdi->execute(['id' => $prodiId]);
        $namaProdi = $stmtNamaProdi->fetchColumn();

        if ($namaProdi === false) {
            http_response_code(422);
            echo json_encode([
                'success' => false,
                'message' => 'Nama program studi tidak
ditemukan'
            ]);
            exit;
        }

        // Pastikan dosen tersedia jika dosen_id diisi
        if ($dosenId !== null) {
            $checkDosen = $pdo->prepare(
                'SELECT id FROM dosen WHERE id = :id'
```

```
            );
            $checkDosen->execute(['id' => $dosenId]);

            if (!$checkDosen->fetch()) {
                http_response_code(422);
                echo json_encode([
                    'success' => false,
                    'message' => 'Dosen tidak ditemukan'
                ]);
                exit;
            }
        }

        // Simpan data dengan prepared statement
        $stmt = $pdo->prepare(
            'INSERT INTO mahasiswa
                (nim, nama, email, prodi, prodi_id, angkatan,
status, dosen_id)
             VALUES
                (:nim, :nama, :email, :prodi, :prodi_id, :angk
atan, :status, :dosen_id)'
        );

        $stmt->execute([
            'nim' => $nim,
            'nama' => $nama,
            'email' => $email,
            'prodi' => $namaProdi,
            'prodi_id' => $prodiId,
            'angkatan' => $angkatan,
            'status' => $status,
            'dosen_id' => $dosenId
        ]);

        $idBaru = (int) $pdo->lastInsertId();

        http_response_code(201);
        echo json_encode([
            'success' => true,
            'message' => 'Mahasiswa berhasil ditambahkan',
            'data' => [
                'id' => $idBaru,
                'nim' => $nim,
                'nama' => $nama,
                'email' => $email,
                'prodi' => $namaProdi,
                'prodi_id' => $prodiId,
```

```
                'angkatan' => $angkatan,
                'status' => $status,
                'dosen_id' => $dosenId
            ]
        ], JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);

    } else {
        header('Allow: GET, POST');
        http_response_code(405);
        echo json_encode([
            'success' => false,
            'message' => 'Method tidak diizinkan'
        ]);
    }

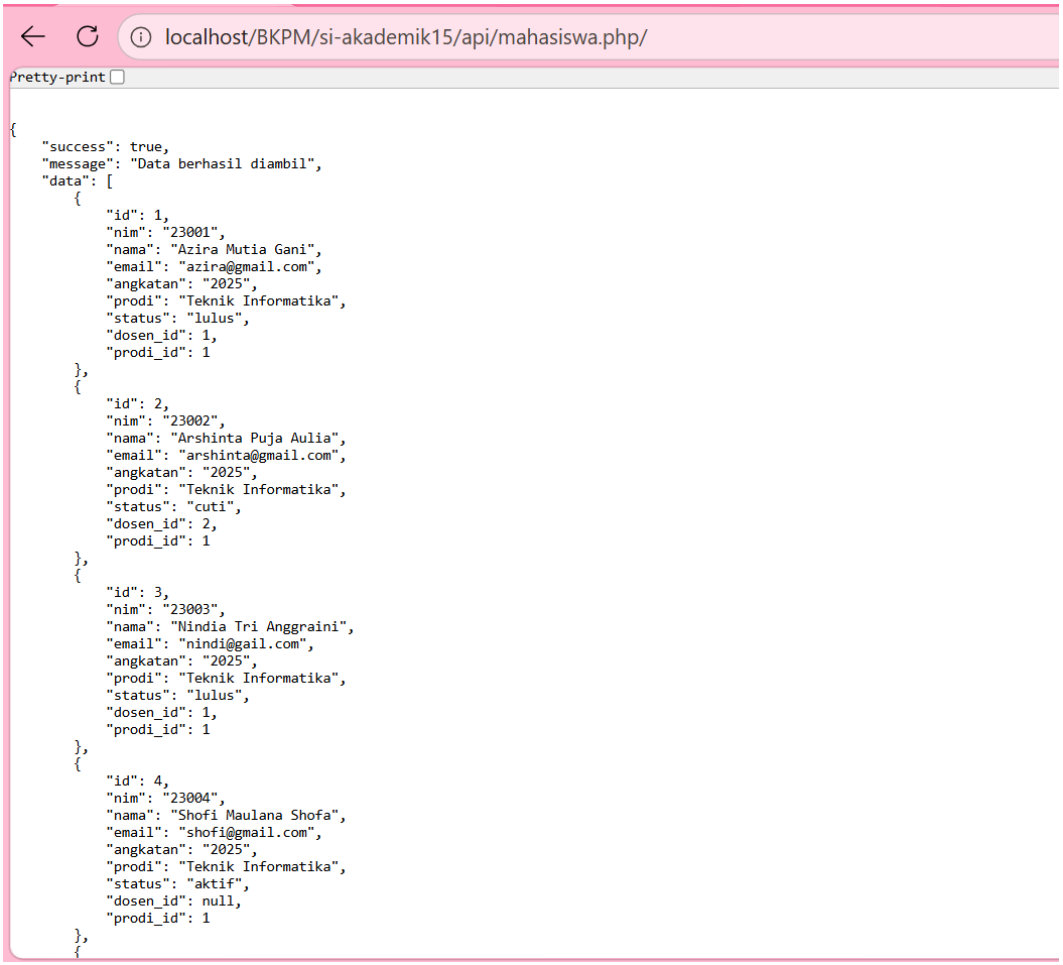
} catch (Throwable $e) {
    $logDir = __DIR__ . '/../storage/logs';
    if (!is_dir($logDir)) {
        @mkdir($logDir, 0775, true);
    }
    error_log(
        '[' . date('Y-m-d H:i:s') . '] API error: '
        . $e->getMessage() . PHP_EOL,
        3,
        $logDir . '/app.log'
    );

    http_response_code(500);
    echo json_encode([
        'success' => false,
        'message' => 'Terjadi kesalahan pada server'
    ]);
}
```

3.1.2 Studi Kasus

Studi kasus dilakukan dengan membuat API untuk mengambil seluruh data mahasiswa menggunakan metode GET. Data yang diperoleh dari database dikirimkan dalam bentuk JSON dengan informasi status dan pesan respons.

Endpoint yang digunakan:
http://localhost/si-akademik15/api/mahasiswa.php

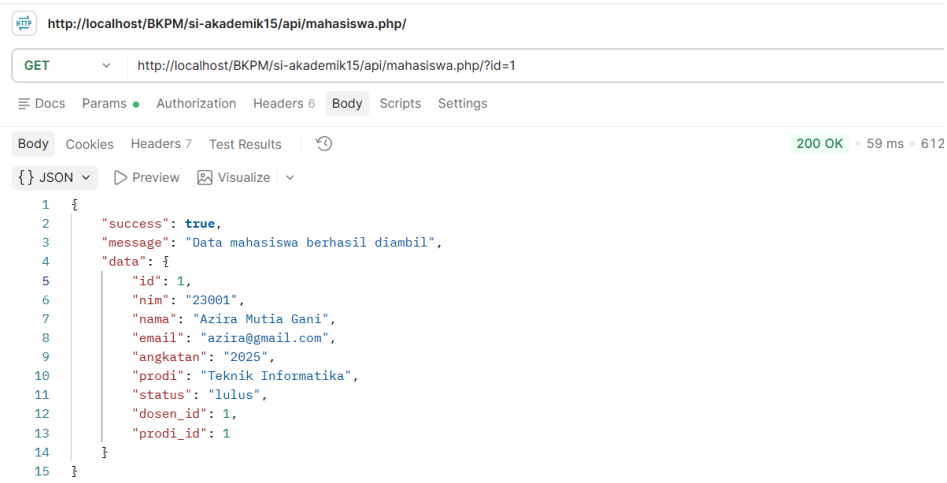


3.1.3 Tugas Pengembangan

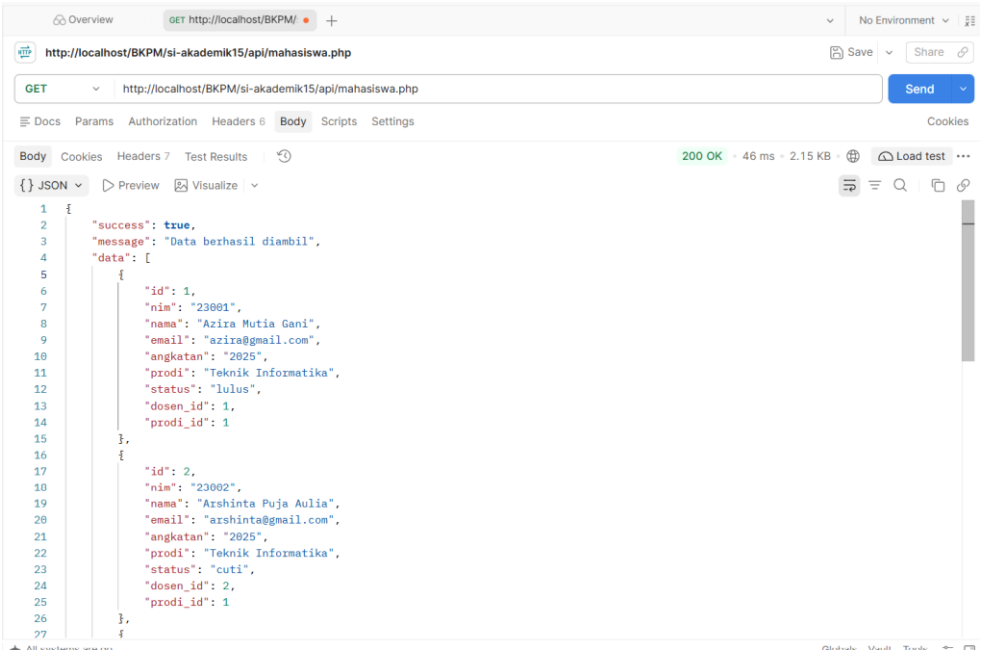
Tugas pengembangan dilakukan dengan menambahkan fitur pengambilan data mahasiswa berdasarkan ID dan penambahan data mahasiswa menggunakan metode POST.

Endpoint yang digunakan:

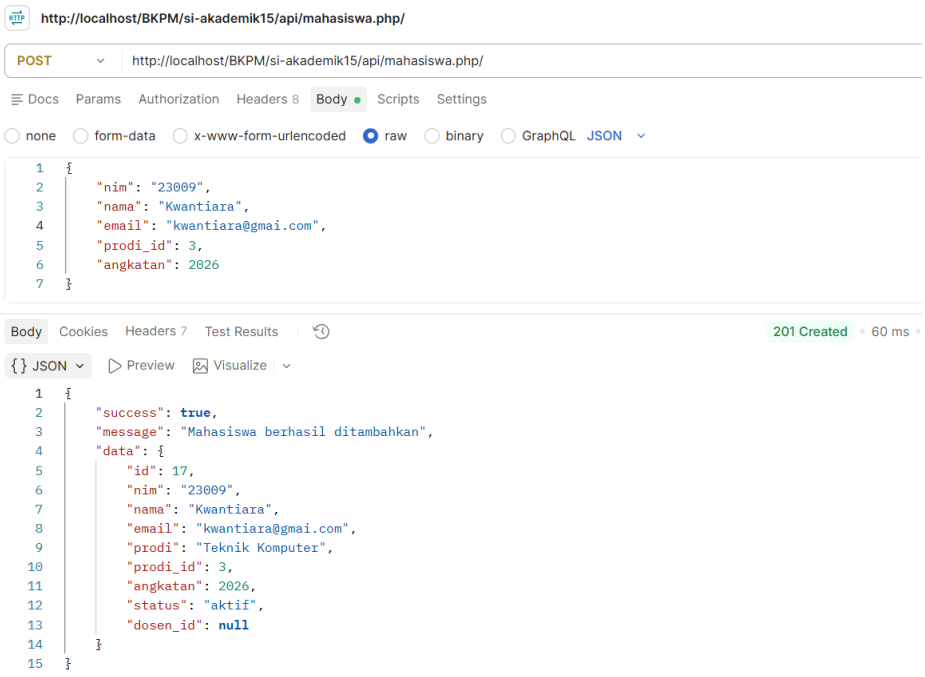
a. GET berdasarkan ID: http://localhost/si-akademik15/api/mahasiswa.php?id=1



b. GET data mahasiswa



c. POST data mahasiswa: http://localhost/si-akademik15/api/mahasiswa.php



Permintaan POST menggunakan format JSON pada bagian request body. Pengujian dilakukan menggunakan Postman untuk memeriksa respons API.

BAB 4. PEMBAHASAN

4.1 Pembahasan

Pada praktikum Acara 15, aplikasi Sistem Informasi Akademik dikembangkan dengan menerapkan konsep API dan JSON untuk pertukaran data mahasiswa. API dibuat menggunakan PHP dan dihubungkan dengan database MySQL. Metode GET digunakan untuk mengambil seluruh data mahasiswa maupun data berdasarkan ID, sedangkan metode POST digunakan untuk menambahkan data mahasiswa baru.

Format JSON digunakan sebagai bentuk respons API agar data dapat dibaca dan diproses oleh aplikasi lain. Penggunaan `json_encode()` membantu mengubah data PHP menjadi JSON, sedangkan `json_decode()` digunakan untuk membaca data JSON yang dikirimkan melalui permintaan POST.

Validasi data diterapkan untuk memastikan informasi mahasiswa yang dikirimkan memenuhi ketentuan. Pemeriksaan NIM duplikat serta validasi data terkait membantu mengurangi kesalahan ketika data disimpan ke database. Selain itu, penggunaan prepared statement mendukung keamanan proses pengolahan data.

Penerapan API dan JSON memungkinkan data mahasiswa diakses melalui endpoint tanpa harus menampilkan data dalam bentuk halaman HTML. Dengan demikian, API dapat menjadi penghubung antara database dan aplikasi yang membutuhkan data mahasiswa.

4.2 Analisis Pengujian

Pengujian dilakukan terhadap endpoint API menggunakan browser dan Postman. Pengujian GET digunakan untuk memeriksa pengambilan seluruh data mahasiswa dan pengambilan data berdasarkan ID. Sementara itu, pengujian POST digunakan untuk memeriksa proses penambahan data mahasiswa melalui request body berformat JSON.

Analisis pengujian dilakukan dengan memperhatikan struktur respons JSON, pesan yang dihasilkan, status code HTTP, serta kesesuaian data dengan database. Pengujian juga perlu mencakup kondisi input tidak valid, NIM duplikat, ID yang tidak ditemukan, dan penggunaan metode HTTP yang tidak didukung.

Apabila ditemukan kesalahan, pemeriksaan dapat dilakukan pada proses pembacaan JSON, validasi input, query database, atau konfigurasi koneksi. Hasil pengujian digunakan untuk mengetahui apakah setiap endpoint telah berjalan sesuai dengan fungsi yang dirancang.

BAB 5. PENUTUP

5.1 Kesimpulan

Praktikum Acara 15 membahas penerapan API dan JSON pada aplikasi Sistem Informasi Akademik menggunakan PHP dan MySQL. Materi yang diterapkan meliputi pengambilan data mahasiswa menggunakan metode GET, penambahan data menggunakan metode POST, serta pengolahan data dalam format JSON.

Penerapan konsep tersebut bertujuan mempermudah pertukaran data antara aplikasi dan server, memastikan data dapat diakses melalui endpoint API, serta membantu proses pengolahan data mahasiswa secara terstruktur. Pengujian menggunakan browser dan Postman juga membantu memeriksa respons API berdasarkan permintaan yang dikirimkan.